



PlugSure CSMS v1.5.0, Deployment Guide

Installation, configuration, first charger, backups, upgrade and operations

What is new since v1.3

- Outside `NODE_ENV=development / test` the processes **refuse to start** without a proper `SECRETS_KEY`, with `OCPP_MIN_SECURITY_PROFILE` below 2, or with `OCPP_AUTO_ADOPT=true`. `NODE_ENV=staging` now counts as production.
- `API_HOST` and the new `OCPP_HOST` default to `127.0.0.1`. New `OCPP_TRUSTED_PROXIES` (default `127.0.0.1,::1`).
- Use `systemctl restart`, never `reload`. Re-deploy `deploy/Caddyfile` (the API host no longer proxies sign-in).
- Row-level security fails closed; the audit log is append-only. Migrations 043–054 (045 unused; 054 is v1.5.0).
- Nightly backup script and systemd timer; OCPP frames kept 90 days, connection attempts 30 days.
- v1.4.1: suspend and resume a charge point. v1.4.2: Scan from live charger fixed, the sidebar shows the installed version, no Start on suspended chargers, corrected deployment docs. v1.4.3: the user menu icons sized correctly. None has migrations or new settings. Go-live gate: `docs/ACCEPTANCE-v1.4.md`.
- v1.4.4: on systemd the database owner credential moves out of `plugsure.env` into a root-only `/etc/plugsure/migrate.env`, read only by the new one-shot `plugsure-migrate.service`, which every API or gateway start runs first (**one configuration change**, section 11.4). A suspended charger stays on the driver map but sells, reserves and queues nothing; suspending releases live reservations; Resume no longer backdates an outage. No migrations, no new application settings.
- v1.5.0: **white-label operator console** (Governance → Console branding). An operator's own console address works only once a **platform administrator approves it** (section 7.5). A driver-app address or identifier already used by another operator now answers 409, not a server error. Migration 054 (additive); no new settings (section 11.1).

Package	<code>plugsure-v1.5.0</code>
Build	git tag <code>v1.5.0</code>
Stack	Node.js 22 · PostgreSQL 16 · Caddy 2 · OCPP 1.6-J, 2.0.1 & 2.1 · OCPI 2.2.1
Verified	v1.5.0: typecheck clean · 829/829 unit and database tests · all 30 end-to-end suites in full mode as <code>plugsure_app</code> , including the new console-brand suite (45 checks) · browser check of the branded console in light and dark themes. v1.4.4: typecheck clean · 819/819 unit and database tests · all 29 end-to-end suites in full mode as <code>plugsure_app</code> · systemd units checked under systemd 255 (migrate before start, credential isolation, failed-migration retry, 100 s migration). v1.4.0: migrations 001–053 applied from an empty PostgreSQL 16 database; all 29 end-to-end suites passing as the restricted runtime role.
Date	2 October 2026
Audience	The operator's deployment engineer, and the platform operations team

Contents

1. Architecture and topology
2. Prerequisites
3. Database and roles
4. Configuration
5. Install, Path A: systemd
6. Install, Path B: Docker
7. TLS and the reverse proxy
8. Commission the first charger
9. Health checks and verification
10. Backups and restore
11. Upgrading from v1.3.0 or a v1.4 release to v1.5.0
12. Operations
13. Troubleshooting
14. Go-live checklist

Path letters. In this guide Path A is **systemd** and Path B is **Docker**. `deploy/README.md` uses the opposite letters (A = Docker Compose, B = systemd). The steps are the same; only the letters differ.

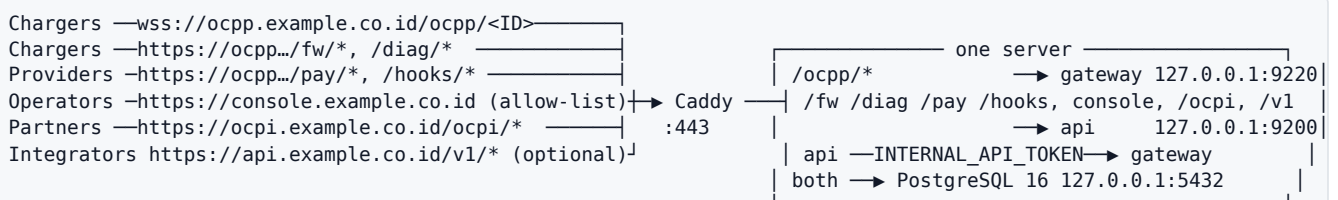
Related documents in the package: `RELEASE-NOTES-v1.4.0.md`, `RELEASE-NOTES-v1.4.1.md`, `RELEASE-NOTES-v1.4.2.md`, `RELEASE-NOTES-v1.4.3.md`, `RELEASE-NOTES-v1.4.4.md`, `RELEASE-NOTES-v1.5.0.md`, `deploy/README.md` (the full runbook), `deploy/AUTEL-HANDOUT.md` (for the charger vendor) and `docs/ACCEPTANCE-v1.4.md` (the hardware acceptance checklist).

1. Architecture and topology

PlugSure runs as two Node.js processes built from the same package, plus PostgreSQL and a Caddy TLS edge. The two processes are separate on purpose: deploying an API fix must never drop live charger WebSockets.

Component	Listens on	Role
OCPP gateway <code>dist/apps/gateway.js</code>	127.0.0.1:9220 (<code>OCPP_HOST</code> , <code>OCPP_PORT</code>)	Charger WebSockets (OCPP 1.6-J, 2.0.1, 2.1). Runs the background workers (<code>RUN_WORKERS=true</code>): load management, compliance, reconciliation, firmware, refunds, outages, webhooks, alert notifications, roaming, retention. Serves <code>/healthz</code> and the token-protected <code>/internal/*</code> bridge.
API + operator console <code>dist/apps/api.js</code>	127.0.0.1:9200 (<code>API_HOST</code> , <code>API_PORT</code>)	REST API (<code>/v1</code>), the console at <code>/</code> , the driver app at <code>/app</code> , the charger-facing firmware download (<code>/fw/*</code>) and log upload (<code>/diag/*</code>), payment and WhatsApp callbacks (<code>/pay/*</code> , <code>/hooks/*</code>), roaming (<code>/ocpi/*</code>).
PostgreSQL 16	127.0.0.1:5432	All data. Loopback only.
Caddy 2	:443, :80 (ACME) 127.0.0.1:8080 (loopback check)	TLS for chargers, operators, partners and integrators. The only internet-facing component.

The API reaches chargers **through the gateway** (the "bridge"): commands over an authenticated internal HTTP call to `GATEWAY_INTERNAL_URL`, and live events relayed through PostgreSQL `NOTIFY` (`EVENT_RELAY=true` on the gateway). Both processes must share the same `INTERNAL_API_TOKEN`.



Caddy host blocks in the supplied `deploy/Caddyfile`:

Host block	Serves
<code>ocpp.example.id</code>	<code>/ocpp/*</code> → gateway; <code>/fw/*</code> , <code>/diag/*</code> , <code>/pay/*</code> , <code>/hooks/*</code> → API. Anything else: 404.
<code>console.example.id</code>	Console and API, behind an office / VPN IP allow-list.
<code>ocpi.example.id</code>	Only <code>/ocpi/*</code> , for roaming partners (token-authenticated, no allow-list). Delete if you do not roam.
<code>api.example.id</code> (optional)	<code>/v1/*</code> , <code>/openapi.json</code> , <code>/api-docs.html</code> with API keys only. Cookies stripped; <code>/v1/auth/*</code> answers 404.

Operator console addresses (optional, v1.5.0)	One block per approved white-label console address, copied from the commented <i>White-label consoles</i> template, each with its own allow-list (section 7.5).
<code>http://127.0.0.1:8080</code>	Loopback listener to the API, for checks on the box.
<code>https://</code> (catch-all, last)	White-label driver apps on operators' own domains; certificates on demand, approved by the API (<code>/d/tls-ask</code>).

A single-process mode (`dist/apps/all-in-one.js`) exists for development. This guide covers the two-process production layout only.

2. Prerequisites

Item	What the sources say
Server	<code>deploy/README.md</code> targets one supervised VM in AWS <code>ap-southeast-3</code> (Jakarta): Ubuntu 24.04 LTS, <code>t3.medium</code> or larger ("2 vCPU / 4 GB is comfortable for one charger and leaves room for Postgres"), 20 GB gp3 root volume, and an Elastic IP so the charger hostname never moves. The OCPP frame log is chatty: plan for growth. The v1.3 guide gave 2 vCPU / 4 GB / 40 GB minimum and 4 vCPU / 8 GB above about 300 chargers; v1.4 sources do not restate or re-test that figure.
Software, Path A (systemd)	Node.js 22 (<code>package.json</code> : <code>"node": ">=22"</code>), PostgreSQL 16, <code>postgresql-client-16</code> (for backups).
Software, Path B (Docker)	Docker Engine with the Compose plugin (<code>docker.io</code> , <code>docker-compose-v2</code>). The image is built on <code>node:22-slim</code> ; the database is the <code>postgres:16</code> image.
Both	Caddy 2, <code>openssl</code> , <code>curl</code> , <code>jq</code> (to read the JSON logs).
DNS	An <code>A</code> record for the charger host (e.g. <code>ocpp.example.co.id</code>) and one for the console host (e.g. <code>console.example.co.id</code>), both to the server's public IP. Optional: <code>ocpi.</code> for roaming, <code>api.</code> for integrators.
Time zone	<code>TZ=Asia/Jakarta</code> . Not cosmetic: WBP/LWBP tariff windows are evaluated in local time.

Package installation on Ubuntu, as given in `deploy/README.md`:

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl postgresql-client-16
# Path A (systemd): Node 22 from NodeSource + local PostgreSQL 16
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs postgresql-16
# Path B (Docker)
sudo apt-get install -y docker.io docker-compose-v2
# Caddy (both paths)
sudo apt-get install -y debian-keyring debian-archive-keyring apt-transport-https
curl -1sLf https://dl.cloudsmith.io/public/caddy/stable/gpg.key \
| sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt \
| sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt-get update && sudo apt-get install -y caddy
```

Firewall

Port (inbound)	Source	Why
443	anywhere	Chargers (<code>wss</code>), console, partners. SIM-card egress IPs are not predictable.
80	anywhere	ACME HTTP-01 challenge for the certificates.
22	your office / admin addresses only	Operations.

```
sudo ufw default deny incoming
sudo ufw allow from <admin-ip> to any port 22 proto tcp
sudo ufw allow 80,443/tcp
sudo ufw enable
```

Never expose 9200, 9220 or 5432. Port 9220 carries plain `ws` and the gateway's `/internal/*` bridge endpoints. In v1.4 both processes bind loopback by default; keep it that way.

Decide the charger hostname before any charger is configured. It is written into every charger's Central System URL. Changing it later means a site visit per unit.

3. Database and roles

PlugSure uses two kinds of database role. Keep them apart.

Role	Used by	Rights
Owner Path A: a dedicated role <code>pluginsure</code> , owner of the database, with <code>CREATEROLE</code> . Path B: the <code>postgres</code> superuser of the container (<code>POSTGRES_PASSWORD</code>).	Migrations only (<code>dist/db/migrate.js</code> / <code>npm run migrate</code>), the seed, and backups. Path A: its connection is kept only in the root-only <code>/etc/pluginsure/migrate.env</code> and <code>backup.env</code> , never in the apps' environment (v1.4.4).	DDL. Creates and provisions the runtime role.
Runtime role <code>pluginsure_app</code> (<code>POSTGRES_APP_USER</code> , default <code>pluginsure_app</code>)	The API and the gateway, always.	Not a superuser, no <code>BYPASSRLS</code> . Row-level security applies to it. No DDL. No <code>UPDATE/DELETE</code> on <code>audit_log</code> .

3.1 How the runtime role gets its password

1. Migrations 002 and 006 create `pluginsure_app` with `LOGIN` and its grants.
2. When `POSTGRES_APP_PASSWORD` is set, the migrator ends by running `ALTER ROLE pluginsure_app LOGIN PASSWORD ...` and `GRANT CONNECT ON DATABASE ...`. It logs *"runtime role provisioned — the applications connect as this, not as the owner"*. Without `POSTGRES_APP_PASSWORD` this step is skipped silently and the app processes cannot log in.
3. The migrator then re-asserts that `audit_log` is append-only for the runtime role.
4. The API and gateway connect with `DATABASE_URL=postgresql://pluginsure_app:<APP_PW>@.../pluginsure`.

3.2 Create the database (Path A)

```
sudo -u postgres createuser pluginsure --createrole --pwprompt # owner; NOT a superuser
sudo -u postgres createdb pluginsure -O pluginsure
```

Server that already has a `pluginsure_app` role (shared cluster, rebuild, earlier install): PostgreSQL 16 lets the owner alter only a role it administers. Grant that once, or migration 006 fails with *permission denied to alter role*:

```
sudo -u postgres psql -c "GRANT pluginsure_app TO pluginsure WITH ADMIN OPTION;"
```

On Path B the `postgres` container creates the database (`POSTGRES_DB`, default `pluginsure`) with UTF-8 and `C` locale, and the `migrate` service does the rest.

3.3 Row-level security fails closed

Migration 048 rewrote every tenant policy to `app_rls_bypass() OR org_id = app_current_org()`. Inside a tenant's request, a missing organisation now matches **nothing** (before v1.4 it matched every row). The same migration makes `audit_log` append-only for the runtime role.

At start-up each process checks the database user:

- As `pluginsure_app` it logs: **row-level security is in force for application queries**. This is the line to look for.

- As a superuser or a role with `BYPASSRLS`, outside `development/test`, it **refuses to start**: *The database user "... is a superuser, so every row-level security policy is ignored ... Connect as `plugsure_app`.*
- If the runtime role holds `UPDATE` or `DELETE` on `audit_log` it logs an error but starts. Revoke them (the next migrator run also does).

4. Configuration

Both paths read the same variables. Path A reads `/etc/plugsure/plugsure.env` (both app units) and, for migrations only, the root-only `/etc/plugsure/migrate.env` (v1.4.4); Path B reads `.env` next to `docker-compose.yml`. Every setting below exists in `.env.example` or `src/config.ts`.

4.1 Generate the secrets

Run once per environment. Store the values in your password manager, away from the backups. Never commit them.

```
openssl rand -hex 32      # AUDIT_HMAC_KEY      keys the audit log's hash chain
openssl rand -hex 32      # SECRETS_KEY         encrypts provider, webhook and roaming secrets and the charger CA key
openssl rand -hex 32      # INTERNAL_API_TOKEN  API ↔ gateway bridge (same value on both)
openssl rand -base64 24    # POSTGRES_APP_PASSWORD runtime role plugsure_app
openssl rand -base64 24    # owner password (role plugsure on Path A, POSTGRES_PASSWORD on Path B)
```

Do not rotate `AUDIT_HMAC_KEY` casually. Every entry written under the old key stops verifying. Export and re-anchor the audit log first. **Changing `SECRETS_KEY`** makes every sealed credential unreadable (payment providers, webhooks, roaming tokens, APNs keys, the charger CA key); they must be entered again.

If a password contains `@` `:` `/` `?` `#` or `%` (base64 output can contain `/` and `+`), percent-encode it inside a connection URL (e.g. `/` → `%2F`).

4.2 Required settings

Outside `NODE_ENV=development` or `test`, any other value (including `staging`) is treated as production.

Setting	Production value	Why / what happens without it
<code>NODE_ENV</code>	<code>production</code>	Enables every production check. The systemd units and the Docker image already set it.
<code>AUDIT_HMAC_KEY</code>	<code>openssl rand -hex 32</code>	Refuses to start if empty: <i><code>AUDIT_HMAC_KEY</code> is empty...</i>
<code>SECRETS_KEY</code>	<code>openssl rand -hex 32</code> (64 hex characters)	Refuses to start if missing, shorter than 32 characters, or the <code>CHANGE_ME</code> placeholder. A longer passphrase is still accepted so existing sealed values keep opening.
<code>OCPD_MIN_SECURITY_PROFILE</code>	2 (or 3)	Gateway refuses to start below 2. Code default is 0 (no charger authentication).
<code>OCPD_AUTO_ADOPT</code>	<code>false</code>	Gateway refuses to start with <code>true</code> . Unknown chargers are parked for an operator.
<code>OCPD_TRUST_PROXY_PROTO</code>	<code>true</code> (TLS at Caddy)	Gateway refuses to start at Profile 2 without its own certificate and without this: <i><code>OCPD_MIN_SECURITY_PROFILE=2</code> requires TLS...</i>
<code>DATABASE_URL</code>	<code>postgresql://plugsure_app:<APP_PW>@127.0.0.1:5432/plugsure</code>	The runtime role. As a superuser the process refuses to start (section 3.3). Path B: compose builds it for you.
<code>POSTGRES_APP_PASSWORD</code>	generated	Read by the migrator to give <code>plugsure_app</code> its password. Path A: in <code>migrate.env</code> , not in <code>plugsure.env</code> (v1.4.4). Path B: compose refuses to start without it.

DATABASE_URL in migrate.env	postgresql://pluginsure:<OWNER_PW>@127.0.0.1:5432/pluginsure	Path A only: the owner connection <code>pluginsure-migrate.service</code> runs migrations with. It lives in the root-only <code>/etc/pluginsure/migrate.env</code> (root:root 0600), never in <code>pluginsure.env</code> , so the running API and gateway do not hold the owner password (v1.4.4; v1.4.3 and earlier used <code>MIGRATION_DATABASE_URL</code> in <code>pluginsure.env</code>).
POSTGRES_PASSWORD	generated	Path B only: the <code>postgres</code> superuser. Compose refuses to start without it.
INTERNAL_API_TOKEN	<code>openssl rand -hex 32</code>	Same value on both processes. Without it the console cannot reach chargers and shows them all offline; the gateway logs a warning. Path B: compose refuses to start without it.
GATEWAY_INTERNAL_URL	<code>http://127.0.0.1:9220</code>	On the API: where the gateway is. Compose sets <code>http://gateway:9220</code> .
EVENT_RELAY	<code>true</code>	On the gateway: live events to the API. Compose sets it.
PUBLIC_BASE_URL	<code>https://ocpp.example.co.id</code>	Firmware and log-upload URLs handed to chargers; payment notification URL shown in the console. Its hostname is one of PlugSure's own addresses, which no operator can take as a console address (section 7.5).
OCPP_PUBLIC_URL	<code>wss://ocpp.example.co.id/ocpp</code>	Printed on commissioning exports and QR codes (without the identity).
OCPI_PUBLIC_URL	<code>https://ocpi.example.co.id</code>	Roaming only. Falls back to <code>PUBLIC_BASE_URL</code> ; with neither, the OCPI API and roaming commands answer 503.
API_TRUSTED_PROXIES	Path A: <code>127.0.0.1</code> Path B: <code>172.16.0.0/12</code> (Docker bridge, as in the v1.3 guide)	Lets the API see the real client IP and HTTPS scheme behind Caddy (rate limits, audit IPs, Secure cookie). Empty means trust nobody. Never set it wider than your proxy.
OCPP_VERSIONS	<code>ocpp1.6</code> for the Autel pilot	Code default <code>ocpp1.6</code> . Add <code>ocpp2.0.1</code> only after a 2.0.1 model has passed acceptance, and <code>ocpp2.1</code> for bidirectional charging or 2.x sandboxes. The Autel handout tells the vendor we advertise only 1.6.
TZ	<code>Asia/Jakarta</code>	Tariff windows and billing months.

4.3 Important optional settings

Setting	Default	Purpose
API_HOST / API_PORT	127.0.0.1 / 9200	API listen address. Was <code>0.0.0.0</code> before v1.4. The image and compose set <code>0.0.0.0</code> inside the container; the port binding decides exposure.
OCPP_HOST / OCPP_PORT	127.0.0.1 outside development / 9220	New in v1.4. Set <code>0.0.0.0</code> only when chargers or a remote API reach the gateway without a local proxy, and then firewall it.
OCPP_TRUSTED_PROXIES	127.0.0.1, ::1 (compose adds 172.16.0.0/12)	New in v1.4. Peers whose <code>X-Forwarded-Proto</code> and <code>X-Client-Cert-Fingerprint</code> the gateway believes. Set it if your TLS terminator does not connect from loopback.
ALLOW_INSECURE_OCPP	false	Acknowledges a supervised bench with Profile < 2 or auto-adapt. Never in production.
POSTGRES_APP_USER	<code>pluginsure_app</code>	Name of the runtime role.

MIGRATION_LOCK_TIMEOUT	10s	How long one migration waits for a table lock before it fails and rolls back. <code>0</code> waits forever.
OCPP_FRAME_RETENTION_DAYS	90	OCPP message log kept this long (hourly pass). <code>0</code> keeps forever.
CONNECTION_ATTEMPT_RETENTION_DAYS	30	Connection-attempt log retention.
API_RATE_LIMIT_PER_MIN	600	Per client IP, for the console and callers without an API key. Kept per API process.
API_KEY_RATE_LIMIT_PER_MIN	600	Per API key, unless the key has its own limit.
API_RATE_LIMIT_SHARED	true outside development/test	Per-key limits kept in PostgreSQL and shared by every API process.
API_KEY_AUTH_FAILURES_PER_MIN	30	Requests per IP per minute with an API key that does not authenticate.
SMTP_ALLOWED_INTERNAL_HOSTS	(none)	New in v1.4. Loopback or private SMTP hosts alert e-mail may use, e.g. a local relay: <code>127.0.0.1,localhost</code> .
RUN_WORKERS	true	Background workers. Exactly one gateway runs them; set <code>false</code> on any extra replica.
STORAGE_DIR	./var/storage	Firmware images and charger logs. Path A unit: <code>/var/lib/plugsure/storage</code> ; image: <code>/var/lib/plugsure</code> .
CONSOLE_PUBLIC_URL	PUBLIC_BASE_URL	Console address put in alert messages. With white-label console addresses (v1.5.0), set it to the shared console address (e.g. <code>https://console.example.co.id</code>): PlugSure's own addresses can never be entered or approved as an operator's console address, and this is the setting that names the shared console (section 7.5).
OFFLINE_ALERT_MINUTES	15	Offline time before a critical charger alert.
OCPP_HEARTBEAT_S	300	Heartbeat interval sent to chargers. Keep proxy timeouts above 3x.
OCPP_KEY_ROTATION_GRACE_MS	86400000	Old AuthorizationKey still accepted for 24 h after rotation.
LOG_LEVEL	info	debug only while chasing a fault.
CSMS_ROOT_CA_FILE	(empty)	Root of the OCPP host's certificate, for a private CA. Empty for Let's Encrypt.
PNC_PKI	none in production	Plug & Charge PKI. <code>mock</code> is refused in production.
MAP_TILE_URL	OpenStreetMap tiles	Point at your own or a commercial tile service before real traffic.
IDLE_FEE_CAP_IDR	100000	Per-session cap on idle fees (a platform safety bound).
PPN_RATE_BPS, PBJT_IN_PPN_BASE, PBJT_BASE	1200, true, energy	Tax treatment. Confirm with a tax adviser before go-live.

v1.4.0 also added settings read outside `src/config.ts` (database pool and timeouts, driver sign-in limits, one-time password lifetime, console idle timeout, a plaintext-secrets escape hatch, `BACKUP_*`). They are listed with their defaults in `RELEASE-NOTES-v1.4.0.md`, "New settings". Mobile carriers put many phones behind one address, so the per-IP driver sign-in limits may need raising.

5. Install, Path A: systemd

1. **Create the service user and install the code.** Unpack the `plugsure-v1.5.0` package (or a checkout of tag `v1.5.0`) into `/opt/plugsure`, then:

```

sudo useradd --system --home /opt/plugsure --shell /usr/sbin/nologin plugsure
cd /opt/plugsure
sudo npm ci # must end "added N packages"
sudo npm run build # tsc -p tsconfig.build.json → dist/
sudo cp -R src/web dist/web && sudo cp -R src/driver-web dist/driver-web # REQUIRED: console and driver app
sudo chown -R plugsure:plugsure /opt/plugsure
sudo install -d -o plugsure -g plugsure /var/lib/plugsure/storage

```

The `cp -R src/web dist/web` step is not optional. The API serves the console from `dist/web`; without it the console answers 404 while `/healthz` looks fine.

2. **Create the database** as in section 3.2.
3. **Create the environment file**, shared by both app units. It holds the runtime role only, never the owner:

```

sudo install -d -m 0750 -o root -g plugsure /etc/plugsure
sudo install -m 0640 -o root -g plugsure /dev/null /etc/plugsure/plugsure.env
sudo -e /etc/plugsure/plugsure.env

```

```

NODE_ENV=production
TZ=Asia/Jakarta
LOG_LEVEL=info
# the API and gateway run as the restricted role (row-level security enforced)
DATABASE_URL=postgresql://plugsure_app:<APP_PW>@127.0.0.1:5432/plugsure
# the owner connection and POSTGRES_APP_PASSWORD do NOT go here: see migrate.env below
AUDIT_HMAC_KEY=<64 hex>
SECRETS_KEY=<64 hex>
INTERNAL_API_TOKEN=<64 hex>
GATEWAY_INTERNAL_URL=http://127.0.0.1:9220
EVENT_RELAY=true
RUN_WORKERS=true
API_HOST=127.0.0.1
API_PORT=9200
OCPP_HOST=127.0.0.1
OCPP_PORT=9220
API_TRUSTED_PROXIES=127.0.0.1
OCPP_MIN_SECURITY_PROFILE=2
OCPP_TRUST_PROXY_PROTO=true
OCPP_AUTO_ADOPT=false
OCPP_VERSIONS=ocpp1.6
PUBLIC_BASE_URL=https://ocpp.example.co.id
OCPP_PUBLIC_URL=wss://ocpp.example.co.id/ocpp
# roaming only
OCPI_PUBLIC_URL=https://ocpi.example.co.id

```

Then the **migration credential file**, root-only and read only by `plugsure-migrate.service` (systemd reads it as root before dropping to `plugsure`):

```

sudo install -m 0600 -o root -g root /dev/null /etc/plugsure/migrate.env
sudo -e /etc/plugsure/migrate.env

```

```

# the database OWNER: migrations need DDL and create/alter plugsure_app
DATABASE_URL=postgresql://plugsure:<OWNER_PW>@127.0.0.1:5432/plugsure
# the same password as in plugsure.env's DATABASE_URL
POSTGRES_APP_PASSWORD=<APP_PW>
# optional: MIGRATION_LOCK_TIMEOUT=10s, PG_STATEMENT_TIMEOUT_MS=0 (a long data migration),
# POSTGRES_APP_USER (only if the runtime role is not plugsure_app)

```

Write values unquoted, one `KEY=value` per line (systemd `EnvironmentFile` syntax).

Owner commands by hand (checks, the pre-deploy dump, troubleshooting). Read the URL out of `migrate.env` rather than sourcing the file as shell code, so a password with `$`, `&` or quotes is safe. It works on local PostgreSQL and RDS alike:

```
OWNER_URL=$(sudo sed -n 's/^DATABASE_URL=//p' /etc/plugsure/migrate.env)
psql "$OWNER_URL" -c "select 1"
# Path B (Docker): docker compose exec postgres psql -U postgres plugsure -c "select 1"
```

Never use `plugsure_app` for these: it is subject to row-level security and sees no rows.

4. **Install the units and migrate.** Three units: the one-shot `plugsure-migrate.service` and the two app units. Running the migrator by hand is optional (every start of the API or gateway also runs it); it runs the same code as `npm run migrate`, as the owner, and provisions `plugsure_app` from `POSTGRES_APP_PASSWORD`:

```
sudo install -o root -g root -m 0644 deploy/plugsure-migrate.service deploy/plugsure-api.service \
  deploy/plugsure-gateway.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl start plugsure-migrate && journalctl -u plugsure-migrate -n 50 --no-pager
# expect: "migrations up to date" and "runtime role provisioned – the applications connect as this, not as the
owner"
```

5. **Create the first administrator.** This is the only command-line user step; everyone else is invited from **Users & Roles**:

```
cd /opt/plugsure && sudo -u plugsure bash -c 'set -a; . /etc/plugsure/plugsure.env; set +a; \
  node dist/db/create-admin.js --email ops.lead@your-cpo.co.id --name "Ops Lead" \
  --org-slug your-cpo --org-name "PT Your CPO Indonesia"'
```

It prints a **one-time password**, shown once. It expires after 72 hours (v1.4.0). Sign in at the console host and choose your own password (12 characters minimum). Re-running the command for the same e-mail resets that user's password. Add `--platform-admin` only for the platform operator's own staff (platform billing and, from v1.5.0, approving console addresses, section 7.5), in the platform operator's own organisation.

6. **Enable and start the units.** Each app unit runs `ExecStartPre+=/usr/bin/systemctl start plugsure-migrate.service` before its own process, so every start or restart migrates first. A failed migration fails that start, and `Restart=always` retries it (for example while the database is still coming up at boot). The gateway is also ordered after the API, and never runs against an old schema:

```
sudo systemctl enable --now plugsure-api plugsure-gateway
journalctl -u plugsure-migrate -n 50 --no-pager # "migrations up to date"
journalctl -u plugsure-api -n 50 --no-pager # "row-level security is in force...", "API bridge engaged", "API
+ console listening"
journalctl -u plugsure-gateway -n 50 --no-pager # "row-level security is in force...", "OCPP gateway listening",
"background workers started"
```

7. **Install the nightly backup** (section 10).
8. **Install the Caddyfile** (section 7).

Restart, never reload. Neither unit has `ExecReload`. Node exits on `SIGHUP`, so a reload would drop every charger. Configuration is read at start: after any change run `sudo systemctl restart plugsure-api plugsure-gateway`. `systemctl reload` now fails with "Job type reload is not applicable".

Unit facts worth knowing: both app units run as `plugsure` with a strict sandbox (`ProtectSystem=strict`; writable only `/var/lib/plugsure`), restart every 5 s (`Restart=always`), and stop retrying after 5 failures in 5 minutes. `InaccessiblePaths=` hides `migrate.env` and `backup.env` from both, so the running API and gateway cannot read the owner credential. Both set `TimeoutStartSec=infinity`, so a long migration (bounded by the migrate unit's own 600 s) is not reported as a failed start while it runs. `plugsure-migrate.service` is not enabled on its own; it also runs as `plugsure`. The gateway gets 90 s to drain sockets on stop, the API 30 s.

6. Install, Path B: Docker

6.1 Docker Compose

1. Unpack and configure:

```
cd /opt/plugsure # the v1.5.0 package
sudo cp .env.example .env && sudo chmod 600 .env
sudo -e .env
```

2. **Edit .env.** Compose refuses to start until `POSTGRES_PASSWORD`, `POSTGRES_APP_PASSWORD`, `AUDIT_HMAC_KEY`, `SECRETS_KEY` and `INTERNAL_API_TOKEN` are set. Also set:

```
OCPP_MIN_SECURITY_PROFILE=2 # .env.example ships 0 – the gateway refuses to start with it
OCPP_AUTO_ADOPT=false # .env.example ships true – the gateway refuses to start with it
OCPP_TRUST_PROXY_PROTO=true
OCPP_VERSIONS=ocpp1.6
PUBLIC_BASE_URL=https://ocpp.example.co.id
OCPP_PUBLIC_URL=wss://ocpp.example.co.id/ocpp
OCPI_PUBLIC_URL=https://ocpi.example.co.id # roaming only
API_TRUSTED_PROXIES=172.16.0.0/12
```

Compose itself sets `NODE_ENV=production`, the `plugsure_app` `DATABASE_URL` (overriding the one in `.env`), `GATEWAY_INTERNAL_URL=http://gateway:9220`, `EVENT_RELAY=true`, `RUN_WORKERS=true`, `API_HOST/OCPP_HOST=0.0.0.0` inside the containers, and `OCPP_TRUSTED_PROXIES=127.0.0.1,::1,172.16.0.0/12`.

3. **Build and start.** Boot order is enforced: `postgres` (healthy) → `migrate` (exits 0) → `gateway` + `api`. The `migrate` service runs as the `postgres` superuser and provisions `plugsure_app`; the apps connect as `plugsure_app`.

```
sudo docker compose build
sudo docker compose up -d
sudo docker compose ps # migrate: Exited (0); postgres, gateway, api: healthy
sudo docker compose logs api | grep 'row-level security' # "row-level security is in force for application queries"
sudo docker compose logs --tail=50 gateway # "OCPP gateway listening", "background workers started"
```

4. **Create the first administrator:**

```
sudo docker compose run --rm api node dist/db/create-admin.js \
  --email ops.lead@your-cpo.co.id --name "Ops Lead" \
  --org-slug your-cpo --org-name "PT Your CPO Indonesia"
```

Host ports are bound to loopback: PostgreSQL `127.0.0.1:5432` (`PG_HOST_PORT`), gateway `127.0.0.1:9220` (`OCPP_BIND`, `OCPP_HOST_PORT`), API `127.0.0.1:9200` (`API_BIND`, `API_HOST_PORT`). Data lives in two named volumes: `plugsure-pgdata` and `plugsure-storage`. Container logs rotate at 20 MB × 5 files.

Do not run `docker compose run --rm migrate node dist/db/seed.js` on production unless you want the demo organisation and site: the seed is for demonstration.

6.2 Plain `docker run` (the CI reference)

The CI `docker` job is a known-good production start from the image: `migrate` as the superuser with `POSTGRES_APP_PASSWORD`, then run both roles as `plugsure_app`. The image runs as a non-root user and sets `NODE_ENV=production` and `TZ=Asia/Jakarta`. Adapted for a server (CI omits the `*_HOST` lines because its runner is disposable; with `--network host` keep them, or the image's `0.0.0.0` binds every host interface):

```

docker build -t plugsure:v1.5.0 .

# 1. migrate as the owner; provisions plugsure_app
docker run --rm --network host \
-e DATABASE_URL=postgresql://postgres:<OWNER_PW>@127.0.0.1:5432/plugsure \
-e POSTGRES_APP_PASSWORD=<APP_PW> \
plugsure:v1.5.0 node dist/db/migrate.js

# 2. API as plugsure_app
docker run -d --name plugsure-api --network host --restart unless-stopped \
-v plugsure-storage:/var/lib/plugsure \
-e DATABASE_URL=postgresql://plugsure_app:<APP_PW>@127.0.0.1:5432/plugsure \
-e AUDIT_HMAC_KEY=<hex> -e SECRETS_KEY=<hex> -e INTERNAL_API_TOKEN=<hex> \
-e GATEWAY_INTERNAL_URL=http://127.0.0.1:9220 -e API_HOST=127.0.0.1 -e API_PORT=9200 \
-e API_TRUSTED_PROXIES=127.0.0.1 \
-e PUBLIC_BASE_URL=https://ocpp.example.co.id -e OCPublic_URL=https://ocpi.example.co.id \
-e OCPP_PUBLIC_URL=wss://ocpp.example.co.id/ocpp \
plugsure:v1.5.0 node dist/apps/api.js

# 3. gateway as plugsure_app, production OCPP settings
docker run -d --name plugsure-gw --network host --restart unless-stopped \
-e DATABASE_URL=postgresql://plugsure_app:<APP_PW>@127.0.0.1:5432/plugsure \
-e AUDIT_HMAC_KEY=<hex> -e SECRETS_KEY=<hex> -e INTERNAL_API_TOKEN=<hex> \
-e OCPP_HOST=127.0.0.1 -e OCPP_PORT=9220 -e EVENT_RELAY=true \
-e OCPP_MIN_SECURITY_PROFILE=2 -e OCPP_TRUST_PROXY_PROTO=true -e OCPP_AUTO_ADOPT=false \
-e OCPublic_URL=https://ocpi.example.co.id -e PUBLIC_BASE_URL=https://ocpp.example.co.id \
-e HEALTHCHECK_URL=http://127.0.0.1:9220/healthz -e HEALTHCHECK_OK_STATUS=200 \
plugsure:v1.5.0 node dist/apps/gateway.js

```

Compose is the supported route; use plain `docker run` only when you need to fit an existing container platform.

7. TLS and the reverse proxy

7.1 Install the Caddyfile

- Copy `deploy/Caddyfile` to `/etc/caddy/Caddyfile` and edit it:
 - replace `ocpp.example.id`, `console.example.id`, `ocpi.example.id` and `api.example.id` with your hostnames (delete the `ocpi.` and `api.` blocks if you do not use them);
 - replace the console allow-list ranges (`203.0.113.0/24 10.0.0.0/8`) with your office / VPN ranges;
 - set the ACME contact (`email ops@example.id`).
- Test with the Let's Encrypt staging CA first.** Uncomment the `acme_ca` line, reload, confirm issuance, then comment it out again and delete the staging certificate. Production Let's Encrypt allows only 5 failed validations per hostname per hour.

```

sudo caddy validate --config /etc/caddy/Caddyfile
sudo systemctl reload caddy
sudo journalctl -u caddy -f          # "certificate obtained successfully"
# after switching back from staging:
sudo rm -rf /var/lib/caddy/.local/share/caddy/certificates
sudo systemctl restart caddy

```

`systemctl reload caddy` is fine: the "restart, not reload" rule applies to the PlugSure units only.

- Confirm the chain a charger will see:

```

openssl s_client -connect ocpp.example.co.id:443 -servername ocpp.example.co.id </dev/null 2>/dev/null \
| openssl x509 -noout -issuer -subject -dates

```

Do not "fix" the validation warning. `caddy validate` prints "Unnecessary header_up X-Forwarded-*". Keep those lines: they *assign* the values Caddy observed, so a client cannot claim TLS it does not have. Keep `header_up X-Client-Cert-Fingerprint` even while no charger uses Profile 3.

7.2 OCPP over wss, Security Profile 2

- Chargers connect to `wss://ocpp.example.co.id/ocpp/<IDENTITY>` on 443. The identity is **always the last path segment**.

- Profile 2 is TLS plus HTTP Basic: username = the charge point identity, byte for byte; password = its AuthorizationKey. Caddy forwards `Authorization` and `Sec-WebSocket-Protocol` unchanged; do not add `header_up` lines for them.
- The gateway learns that TLS was used from Caddy's `X-Forwarded-Proto`. That needs `OCPP_TRUST_PROXY_PROTO=true` and Caddy connecting from an address in `OCPP_TRUSTED_PROXIES`.
- Use `handle` with a path matcher for `/ocpp/*`, never `handle_path`, `uri strip_prefix`, `rewrite` or a redirect: they mangle the identity.
- The supplied timeouts are 900 s (three missed 300 s heartbeats). Any proxy or load balancer in the path must allow at least that, or healthy sockets drop every few minutes.
- Chargers need the root of the server certificate in their trust store: **ISRG Root X1** for Let's Encrypt.

7.3 Trusted proxies

Layer	Setting	Value with Caddy on the same host
Caddy	(none)	Caddy is the internet edge and trusts no proxy by default. Add <code>trusted_proxies</code> only if an ALB or CloudFront is ever put in front of it, naming only that proxy.
Gateway	<code>OCPP_TRUSTED_PROXIES</code>	<code>127.0.0.1, ::1</code> (default). Compose: also <code>172.16.0.0/12</code> .
API	<code>API_TRUSTED_PROXIES</code>	<code>127.0.0.1</code> . Docker: <code>172.16.0.0/12</code> .

7.4 Security Profile 3 (optional)

Download PlugSure's charging-station CA (Operate → Onboarding → Certificate authority) to `/etc/caddy/plugsure-charger-ca.pem`, uncomment the `tls { client_auth ... }` block in the OCPP host and reload Caddy. Keep `OCPP_TRUST_PROXY_PROTO=true`. The Autel pilot uses Profile 2 only.

7.5 White-label console addresses (v1.5.0, optional)

An operator can brand its console in **Governance → Console branding** (product name, tagline, colours, logo; needs `org:write`). Branding alone needs no server change: every user of that operator sees it once signed in, on any console address. Giving the console an **address of its own**, such as `console.nusantaracharge.id`, takes four steps:

1. **The operator enters the address** under **Console web address** and saves. It shows as *waiting for approval*. Until it is approved it has **no effect**: the address shows PlugSure's sign-in page and restricts nobody, and a waiting claim blocks no one else. So an operator cannot claim the shared console's name or another operator's.
2. **The operator creates a DNS record** for the name pointing at this server (A `<ELASTIC_IP>`, or a CNAME to the shared console host), before you reload Caddy, so the certificate can be issued.
3. **Add a Caddy site block**. Copy the commented *White-label consoles* template in `deploy/Caddyfile` once per address. Set the hostname and that operator's office or VPN ranges in its own allow-list, then validate and reload; Caddy obtains the certificate on reload:

```
sudo -e /etc/caddy/Caddyfile          # uncomment a copy of the template: hostname + @notoffice ranges
sudo caddy validate --config /etc/caddy/Caddyfile
sudo systemctl reload caddy
```

4. **A platform administrator approves it** on the same page, under **Governance → Console branding → Console web addresses**, or with the API (the body names the address the operator has now, so an approval cannot land on one changed meanwhile):

```
curl -sS -X POST http://127.0.0.1:9200/v1/platform/console-hostnames/<orgId>/approve \
-H "Authorization: Bearer $PLATFORM_KEY" -H 'Content-Type: application/json' \
-d '{"hostname": "console.nusantaracharge.id"}'
# list addresses, approved or waiting: GET /v1/platform/console-hostnames; withdraw: POST .../<orgId>/revoke
```

Only one operator can have a given address approved; one already approved for another console or used by a driver app is refused with 409.

- **Once approved**, the sign-in page on that address shows the operator's brand, and **only that operator's accounts can sign in**. Any other account is refused exactly as a wrong password is: the same answer, counted as a failed attempt, no session.

- **Changing the address withdraws the approval**; the platform administrator can also withdraw it. **Back to PlugSure** removes the brand, and the address then accepts any account again.
- Console addresses are **deliberately not served by the on-demand https:// catch-all**: that block serves only driver apps, and a console must keep its allow-list. Without its own site block the address does not reach the console.
- **PlugSure's own addresses are reserved**: the hostnames of `PUBLIC_BASE_URL`, `CONSOLE_PUBLIC_URL`, `API_PUBLIC_URL`, `DRIVER_PUBLIC_URL`, `OCPI_PUBLIC_URL` and `OCPP_PUBLIC_URL` can never be entered or approved. Make sure the shared console address is among them: set `CONSOLE_PUBLIC_URL` to it (section 4.3). Keep `PUBLIC_BASE_URL` on the OCPP host, because chargers download firmware from it and the console host is behind an allow-list. Any other shared hostname, such as a second one for portal users, is protected by the approval itself: never approve it for an operator.

Creating a platform administrator. Approving needs `platform:admin`: an account created with `--platform-admin` in the platform operator's own organisation, never a customer's (`src/db/create-admin.ts`). From a source checkout:

```
npm run create-admin -- --email ops@plugsure.example --name "Platform Ops" \
  --org-slug plugsure --org-name "PlugSure" --platform-admin
```

On a built install use `node dist/db/create-admin.js` with the same arguments, as in section 5, step 5 (Path A) or 6.1, step 4 (Path B: `docker compose run --rm api node dist/db/create-admin.js ...`). `--org-name` is needed only when the organisation does not exist yet. The one-time password is printed once.

8. Commission the first charger

A production gateway refuses unknown identities (`OCPP_AUTO_ADOPT=false`) and demands Basic credentials over TLS (Profile 2). So the identity and its key must exist **before** the charger dials in. You can do this in the console (**Onboarding** → **Add charge point**, which generates the key and shows it once) or with the API, as below.

Set `PLUGSURE_KEY` to an operator API key (issued from the console, or with `POST /v1/api-keys`; a key only carries permissions its issuer holds). Run on the server, against the loopback API.

1. **Register the identity**, exactly as the vendor will configure it (case-sensitive; letters, digits and `. _ : -` only):

```
curl -sS -X POST http://127.0.0.1:9200/v1/charge-points \
  -H "Authorization: Bearer $PLUGSURE_KEY" -H 'Content-Type: application/json' \
  -d '{"ocppIdentity": "AUTEL-DC60-SMB-002", "siteId": "<site-uuid>"}'
```

The charger is created in `pending_adoption`. If it already dialled in and was refused, it is in the adoption queue: use `POST /v1/pending-chargers/<identity>/adopt` with the same body instead. Since v1.4.0, registering an identity that has already tried to connect needs a **platform** administrator (a tenant gets `409 identity_needs_platform_approval`).

2. **Issue the AuthorizationKey**. It is returned once. Write the same value into the charger (Autel: *Settings* → *OCPP* → *Authorization Key*):

```
curl -sS -X POST http://127.0.0.1:9200/v1/charge-points/AUTEL-DC60-SMB-002/authorization-key \
  -H "Authorization: Bearer $PLUGSURE_KEY"
```

3. **Set Security Profile 2** (a `PUT`):

```
curl -sS -X PUT http://127.0.0.1:9200/v1/charge-points/AUTEL-DC60-SMB-002/security-profile \
  -H "Authorization: Bearer $PLUGSURE_KEY" -H 'Content-Type: application/json' \
  -d '{"profile": 2}'
```

4. **Activate it**. Only now does BootNotification answer `Accepted`:

```
curl -sS -X POST http://127.0.0.1:9200/v1/charge-points/AUTEL-DC60-SMB-002/activate \
  -H "Authorization: Bearer $PLUGSURE_KEY"
```

5. **Send the vendor** `deploy/AUTEL-HANDOUT.md` with the placeholders filled in (URL, identity, support contact). Send the AuthorizationKey **by a separate channel**. The handout tells Autel to configure: OCPP 1.6j only, Central System URL `wss://<ocpp host>/ocpp/{identity}`, Security Profile 2, HeartbeatInterval 300 s, MeterValueSampleInterval 60 s, ISRG Root X1 in the trust store.

6. **Watch the connection attempts** while the installer power-cycles the unit: `GET /v1/connection-attempts`, or **Connections** in the console. Every attempt is recorded with its outcome, TLS state, offered subprotocols and whether credentials were sent.

A BootNotification answered **Pending** is not an error: the charger reached us and authenticated, but is not activated yet, or it is **suspended** (v1.4.1). Rotate the AuthorizationKey after commissioning: old and new keys both work for 24 h.

Suspend and resume (v1.4.1). Console: open the charger, **Suspend...**, optional reason; **Resume** on the banner. API: `POST /v1/charge-points/:identity/suspend` (optional `{"reason":"..."}`) and `POST /v1/charge-points/:identity/resume`, both need `charge_point:write` and answer 409 when the current state does not allow it. While suspended the charger keeps its key and stays connected, new cards and starts are refused, and a running session finishes and is billed normally.

- **Driver app (v1.4.4):** the charger stays on the map, its connectors shown **Unavailable**, but it sells, reserves and queues nothing: quotes, payments, checkouts and reservations are refused, and it is never offered to the next driver in a queue. A charge paid before the suspension is not started; the unused payment is refunded.
- **Suspending** releases live driver and roaming (OCPI) reservations on the charger (CancelReservation; the fee is waived or refunded, never a no-show) and puts queue offers back to waiting, so drivers keep their place. The response reports `reservationsReleased`. It also closes any open outage: a suspension is planned downtime.
- **Resume** marks a connected charger online at once; otherwise it is offline from the moment of resume, not backdated to before the suspension.

9. Health checks and verification

9.1 On the server

```
curl -s http://127.0.0.1:9200/healthz | jq      # API: {"ok":true,"db":true,"connectedChargePoints":0,"time":"..."}
curl -s http://127.0.0.1:9220/healthz | jq    # gateway: {"ok":true,"connected":0,"time":"..."} (503 if the database
is unreachable)
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:9220/      # 426: the listener only upgrades to WebSocket
curl -sI http://127.0.0.1:8080/healthz      # through Caddy's loopback listener
```

9.2 From outside

```
curl -sI https://ocpp.example.co.id/ocpp/test | head -1      # 426 Upgrade Required = TLS edge + gateway reachable
curl -sI https://ocpp.example.co.id/healthz | head -1      # 404: the OCPP host serves only /ocpp /fw /diag /pay
/hooks
curl -sI https://console.example.co.id/ | head -1          # 200 from an allow-listed network, 403 elsewhere
curl -sI https://api.example.co.id/v1/auth/login | head -1 # 404 if you kept the api. host: sign-in is not served
there
```

`/healthz` is not routed on the OCPP host, so check health on the server as in 9.1 and use the 426 above as the external check. The Hardware Acceptance Test (step 0, **Health**) does the same.

9.3 Start-up log lines

Line	Process	Means
migrations up to date	migrator (Path A: <code>journalctl -u plugsure-migrate</code>)	Schema current (001-054; 045 unused).
runtime role provisioned – ...	migrator	<code>plugsure_app</code> has its password.
row-level security is in force for application queries	both	Connected as the runtime role. Required.
OCPP gateway listening with <code>tls: "expected at proxy"</code> , <code>minSecurityProfile: 2</code> , <code>autoAdopt: false</code>	gateway	Production OCPP posture.
background workers started: ...	gateway	Workers running (<code>RUN_WORKERS=true</code>).
API bridge engaged: ...	API	API reaches chargers through the gateway.
API + console listening	API	API up.

9.4 Audit chain

```
curl -s http://127.0.0.1:9200/v1/audit -H "Authorization: Bearer $PLUGSURE_KEY" | jq .chain
# expect "ok": true and "problems": [] (needs the audit:read permission)
```

In the console, the same check is **Govern → Audit → Verify**. Then sign in, confirm the live indicator is green and the Dashboard loads.

Never run the end-to-end suites (`tools/e2e/*`) against production. They create test sites, chargers and users.

10. Backups and restore

`tools/backup/pg-backup.sh` writes a `pg_dump -Fc` of the whole database and a tar of `STORAGE_DIR` (firmware images and charger logs are not in the database). It checks the dump is readable, writes SHA-256 sums, optionally copies everything off the host, and only then deletes files older than `BACKUP_RETENTION_DAYS`. A failed run never deletes the last good copy and exits non-zero.

Variable (<code>/etc/plugsure/backup.env</code> , <code>root:root 0600</code>)	Default	Notes
<code>BACKUP_DATABASE_URL</code>	(required)	The owner (or a role with <code>BYPASSRLS</code>). Never <code>plugsure_app</code> : under RLS it would dump only what an unscoped session sees.
<code>BACKUP_DIR</code>	<code>/var/backups/plugsure</code>	
<code>STORAGE_DIR</code>	<code>/var/lib/plugsure/storage</code>	<code>''</code> to skip.
<code>BACKUP_RETENTION_DAYS</code>	14	
<code>BACKUP_OFFSITE_CMD</code>	(none)	e.g. <code>aws s3 sync "\$BACKUP_DIR" s3://<bucket>/plugsure/ --only-show-errors</code> . A backup on the same disk is not a backup.

10.1 Install the timer (Path A)

```
sudo install -o root -g root -m 0755 tools/backup/pg-backup.sh /opt/plugsure/tools/backup/pg-backup.sh
sudo install -o root -g root -m 0644 deploy/plugsure-backup.service deploy/plugsure-backup.timer
/etc/systemd/system/
sudo install -d -o plugsure -g plugsure -m 0700 /var/backups/plugsure
sudo install -o root -g root -m 0600 /dev/null /etc/plugsure/backup.env # then edit; root-only: it holds the owner
credential (v1.4.4)
sudo systemctl daemon-reload && sudo systemctl enable --now plugsure-backup.timer
sudo systemctl start plugsure-backup && journalctl -u plugsure-backup -n 50 # run once by hand
```

The timer runs nightly at 02:15 local time (up to 10 minutes random delay), and runs a missed backup at the next boot. Alert on a failed unit (`systemctl --failed`). The timer is shipped but not installed by default.

Path B (Docker): run the script from the host against the published PostgreSQL port with the superuser in `BACKUP_DATABASE_URL`, set `STORAGE_DIR=''`, and archive the storage volume:

```
docker run --rm -v plugsure-storage:/s -v /var/backups/plugsure:/b debian tar czf /b/storage-$(date +%F).tar.gz -C /s .
```

10.2 Restore drill

Rehearse monthly and after every major upgrade, on a non-production host. These are the real recovery commands:

```
# 1. verify the newest backup
cd /var/backups/plugsure && sha256sum -c plugsure-<stamp>.sha256
# 2. restore into an empty database (roles are cluster-wide and not in the dump)
sudo -u postgres psql -c "CREATE ROLE plugsure NOLOGIN" 2>/dev/null || true
sudo -u postgres psql -c "CREATE ROLE plugsure_app NOLOGIN" 2>/dev/null || true
sudo -u postgres createdb plugsure_restore -O plugsure
time pg_restore --exit-on-error --jobs=4 -d plugsure_restore plugsure-db-<stamp>.dump
# 3. check it is complete and current
psql -d plugsure_restore -c "select max(name) from schema_migration"
psql -d plugsure_restore -c "select count(*), max(started_at) from charging_session"
psql -d plugsure_restore -c "select has_table_privilege('plugsure_app','charging_session','SELECT')"
# 4. point a scratch API at it (plugsure_app, the SAME AUDIT_HMAC_KEY and SECRETS_KEY): Govern → Audit → Verify
# 5. restore the storage and spot-check a firmware image
tar -xzf plugsure-storage-<stamp>.tar.gz -C /tmp/restore-check
# 6. record how long steps 2-5 took: that is your recovery time
sudo -u postgres dropdb plugsure_restore
```

Real recovery: stop both units, restore into a fresh `plugsure` database, restore `STORAGE_DIR`, start `plugsure-api` (migrations bring an older dump up to date), then `plugsure-gateway`.

A restore without the original `SECRETS_KEY` loses every sealed provider credential; without the original `AUDIT_HMAC_KEY` the audit chain no longer verifies. Keep both keys in a secrets store separate from the backups.

11. Upgrading from v1.3.0 or a v1.4 release to v1.5.0

11.1 From v1.4.4 to v1.5.0

One additive migration (**054**) and **no new settings**. Back up (11.2, step 1), deploy the new build and restart; migration 054 applies on start:

- **Path A (systemd):** `npm ci && npm run build`, copy `src/web` and `src/driver-web` into `dist/`, `chown`, then `systemctl restart plugsure-api plugsure-gateway`. `plugsure-migrate.service` applies 054 first; `journalctl -u plugsure-migrate` shows “migrations up to date”. No unit or environment-file change.
- **Path B (Docker):** `docker compose build && docker compose up -d`. The migrate service applies 054 before the apps start.

Migration 054 adds the `console_brand` table, with row-level security in the post-048 form (`app_rls_bypass()` OR `org_id = app_current_org()`) and grants for `plugsure_app`. It needs no data change. **Nothing changes until an operator sets a brand:** every console looks as before. Console addresses of operators’ own need the Caddy and approval steps in section 7.5; branding alone needs no server change.

```
# Path A
cd /opt/plugsure && sudo npm ci && sudo npm run build
sudo cp -R src/web dist/web && sudo cp -R src/driver-web dist/driver-web
sudo chown -R plugsure:plugsure /opt/plugsure
sudo systemctl restart plugsure-api plugsure-gateway
journalctl -u plugsure-migrate -n 20 --no-pager # "migrations up to date"
# Path B
sudo docker compose build && sudo docker compose up -d
```

11.2 From v1.3.0 to v1.5.0

1. **Back up** as the owner, and note the file name:

```
# Path A, v1.4.4 layout (see 11.4); the directory is created once and the nightly backup uses it too
sudo install -d -o plugsure -g plugsure -m 0700 /var/backups/plugsure
OWNER_URL=$(sudo sed -n 's/^DATABASE_URL=//p' /etc/plugsure/migrate.env)
sudo -u plugsure pg_dump "$OWNER_URL" -Fc \
-f /var/backups/plugsure/plugsure-predeploy-$(date +%F-%H%M).dump
```

Before the move to `migrate.env` (11.4), take `OWNER_URL` from wherever your owner connection is kept today. Path B: run the backup script with the `postgres` superuser as in section 10. Also archive `STORAGE_DIR` (or run `tools/backup/pg-backup.sh` once).

2. **Check the settings** in `plugsure.env` / `.env` before you restart anything. Each of these now stops start-up:

- `SECRETS_KEY` set, not the placeholder, at least 32 characters (new deployments: 64 hex). If you must change it, every sealed credential has to be entered again.
- `OCPP_MIN_SECURITY_PROFILE=2` (or 3) and `OCPP_AUTO_ADOPT=false` on the gateway.
- `NODE_ENV=staging` (or anything but `development/test`) is now production.

3. Review the new behaviour:

- Roaming needs `OCPI_PUBLIC_URL` (or `PUBLIC_BASE_URL`); otherwise the OCPI API answers 503.
- systemd: `API_HOST` and `OCPP_HOST` now default to `127.0.0.1`. Fine with Caddy on the same host; if anything reached 9200 or 9220 directly from another machine, it no longer can.
- Set `OCPP_TRUSTED_PROXIES` if your TLS terminator does not connect from loopback.

4. Replace the code with the v1.5.0 package (keep the environment file), then:

- Path A: `npm ci && npm run build`, copy `src/web` and `src/driver-web` into `dist/`, make the v1.4.4 configuration change (11.4: owner credential into `migrate.env`), **install all three units** from `deploy/` (`plugsure-migrate.service` is new; the gateway's old `ExecReload` is gone), install the backup units with a root-only `backup.env` (section 10), `systemctl daemon-reload`, then `systemctl restart plugsure-api plugsure-gateway plugsure-migrate.service`, started from each unit's `ExecStartPre`, applies migrations 043-054.
- Path B: `docker compose build && docker compose up -d`. The `migrate` service applies them.

5. Re-deploy `deploy/Caddyfile` (keep your hostnames, allow-list and e-mail): the `api.` host no longer proxies operator sign-in. `caddy validate`, then `systemctl reload caddy`.

6. Verify as in section 9, including the RLS log line and `GET /v1/audit`.

11.3 Migrations 043-054

All additive and forward-only (045 is unused). 043-053 came with v1.4.0; 054 with v1.5.0. Each runs in its own transaction with `MIGRATION_LOCK_TIMEOUT` (10 s); the whole run holds an advisory lock, so two migrators never run at once.

File	Effect
<code>043_session_start_register</code>	An unknown start register is recorded as unknown, not zero.
<code>044_payments_refund_bounds</code>	A refund can never exceed what was captured (database CHECK).
<code>046_driver_auth_limits</code>	Rate-limit counters for driver sign-in steps.
<code>047_ocpi_trust</code>	Roaming trust boundaries; partner CDRs can be held for review.
<code>048_rls_fail_closed_audit_append_only</code>	RLS fails closed; <code>audit_log</code> append-only.
<code>049_prepaid_paid_at</code>	The prepaid claim window runs from payment.
<code>050_charger_trust_hardening</code>	Charger provisioning and PKI hardening.
<code>051_billing_effective_dates</code>	Billing facts captured when they happen (e.g. the fleet at session time).
<code>052_ops_rate_buckets_retention</code>	Shared per-key rate limiter; retention support.
<code>053_console_account_hardening</code>	One-time password expiry, console idle timeout, case-insensitive unique e-mail.
<code>054_console_brand</code> (v1.5.0)	The <code>console_brand</code> table for the white-label operator console, with row-level security; one approved console address per operator.

Check e-mail duplicates before upgrading from v1.3.0. If two users' e-mails differ only in letter case, 053 skips the unique index `app_user_email_lower_key`. It says so only in a PostgreSQL `NOTICE`, which the migrator log may not show. Check first, as the owner:

```
psql "$OWNER_URL" -c "SELECT lower(email), array_agg(id) FROM app_user GROUP BY 1 HAVING count(*) > 1"
# Path B: sudo docker compose exec postgres psql -U postgres plugsure -c "..."
```

Resolve any rows it returns. If you upgraded with duplicates, resolve them and then run `CREATE UNIQUE INDEX app_user_email_lower_key ON app_user (lower(email));`. If a migration fails on its lock timeout, nothing was applied: find the blocker in `pg_stat_activity` and restart the API (or re-run `migrate`). After the upgrade the retention pass works off the old OCPP frame backlog over several hours, in batches.

11.4 From v1.4.0-v1.4.3 to v1.5.0

Migration 054 applies on start, as in 11.1; there are no new application settings. Back up (11.2, step 1), deploy the new build, restart. The v1.4.4 systemd change below still applies when coming from before v1.4.4. **Path B (Docker) needs no configuration change. Path A (systemd) must make one configuration change:** the owner credential leaves `pluginsure.env`, which both app processes load, for a root-only `migrate.env` read only by the new `pluginsure-migrate.service`.

1. Create `/etc/pluginsure/migrate.env`, root:root 0600, with `DATABASE_URL=` the owner connection (your old `MIGRATION_DATABASE_URL` value) and `POSTGRES_APP_PASSWORD=` the `pluginsure_app` password (section 5, step 3).
2. Remove `MIGRATION_DATABASE_URL` and `POSTGRES_APP_PASSWORD` from `/etc/pluginsure/pluginsure.env`. If the nightly backup is installed, make `/etc/pluginsure/backup.env` root:root 0600: it holds the owner credential too.
3. Build, then install `pluginsure-migrate.service` and the updated `pluginsure-api.service` and `pluginsure-gateway.service` into `/etc/systemd/system/`.
4. `systemctl daemon-reload`, then `systemctl restart pluginsure-api pluginsure-gateway`.

```
# Path A
sudo install -m 0600 -o root -g root /dev/null /etc/pluginsure/migrate.env
sudo -e /etc/pluginsure/migrate.env # DATABASE_URL=<owner>, POSTGRES_APP_PASSWORD=<app pw>
sudo -e /etc/pluginsure/pluginsure.env # delete MIGRATION_DATABASE_URL and POSTGRES_APP_PASSWORD
# only if the nightly backup is installed:
sudo chown root:root /etc/pluginsure/backup.env && sudo chmod 0600 /etc/pluginsure/backup.env
cd /opt/pluginsure && sudo npm ci && sudo npm run build
sudo cp -R src/web dist/web && sudo cp -R src/driver-web dist/driver-web
sudo chown -R pluginsure:pluginsure /opt/pluginsure
sudo install -o root -g root -m 0644 deploy/pluginsure-migrate.service \
    deploy/pluginsure-api.service deploy/pluginsure-gateway.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl restart pluginsure-api pluginsure-gateway
journalctl -u pluginsure-migrate -n 20 --no-pager # "migrations up to date"
# Path B
sudo docker compose build && sudo docker compose up -d
```

Coming from before v1.4.2 with an old environment file. Earlier copies of `deploy/README.md` showed `DATABASE_URL` as the database owner. If your apps connect as the owner, set `DATABASE_URL` in `pluginsure.env` to `pluginsure_app` and put the owner connection in `migrate.env`, as in section 5, step 3, before restarting.

11.5 Behaviour to brief operators on

- Suspended chargers boot `Pending` and refuse new cards; decommissioned ones are refused at connect. A remote start on a suspended, unadopted or decommissioned charger answers 409 with the reason (v1.4.1).
- A suspended charger stays on the driver map but sells, reserves and queues nothing (the operator QRIS checkout also answers 409). Suspending releases live driver and roaming reservations (fee waived or refunded) and puts queue offers back to waiting; a driver put back keeps the original join time, so on a site with a short maximum wait their place can expire soon after. Resume marks a connected charger online at once, otherwise offline from now (v1.4.4).
- A prepaid QR works only on the connector it was paid for.
- Unsolicited certificate requests, and anything below Profile 2, are refused.
- One-time passwords expire after 72 h; changing a password signs out other sessions; console sessions have an idle timeout.
- Webhook receivers must answer within 15 s.
- Plug & Charge refuses when revocation cannot be checked (organisations that saved `acceptWhenOcspUnavailable: true` keep it).
- **Console branding (v1.5.0):** Governance → Console branding (needs `org:write`) sets the product name, tagline, colours and logo every user of the operator sees, including its site-owner and fleet portals. A console address of its own works only after the platform administrator adds its Caddy block and approves it (section 7.5); then only the operator's accounts sign in there. Changing the address withdraws the approval. A developer sandbox cannot be branded.
- Roaming: a partner that is both CPO and eMSP needs two connections; a hub acts for no clients until it sends `HubClientInfo`; held partner CDRs are at `GET /v1/roaming/cdrs/held`.

11.6 Rollback

Roll back **code**, never the schema: migrations are forward-only. If a migration itself is the problem, restore the pre-upgrade dump.

```
# Path A  restore the previous package into /opt/plugsure, rebuild, copy web assets, systemctl restart both units
# Path B  PLUGSURE_TAG=<previous-tag> docker compose up -d --no-deps gateway api
```

Rolling back from v1.5.0 to v1.4.4: the `console_brand` table from migration 054 stays (migrations are forward-only) and v1.4.4 does not use it, which is harmless; operators' branding and console addresses simply stop applying until v1.5.0 is back. Remove any white-label console site blocks from Caddy if you do not plan to return.

Rolling back from v1.4.4 to v1.4.0–v1.4.3 on Path A: keep the v1.4.4 unit files and `/etc/plugsure/migrate.env`. Older releases run the same `dist/db/migrate.js`, which reads `DATABASE_URL` from `migrate.env`, so nothing goes back into `plugsure.env`. Do not reinstall the older unit files: they expect `MIGRATION_DATABASE_URL` in `plugsure.env`.

12. Operations

Logs

Everything is single-line JSON (pino). Levels: 50 error, 40 warn, 30 info, 20 debug. Credentials are redacted; card IDs, eMAIDs and phone numbers are masked.

```
# Path A
journalctl -u plugsure-gateway -f
journalctl -u plugsure-migrate -n 50 --no-pager # the migrator run before each start
journalctl -u plugsure-api --since "30 min ago" -o cat | jq -c 'select(.level >= 40)'
# Path B
docker compose logs -f gateway
docker compose logs --since 30m api | jq -c 'select(.level >= 40)'
# TLS edge
journalctl -u caddy -f
tail -f /var/log/caddy/ocpp-access.log | jq -c '{ts,status,uri:.request.uri,ip:.request.remote_ip}'
```

The richest fault-finding tool is the OCPP frame log: **OCPP log** in the console, or `GET /v1/charge-points/<identity>/frames.ndjson`. The gateway warns on its own health (`gateway health`, `gateway paused`, `slow worker passes`).

Restart, not reload

Configuration is read at start. After any change: `systemctl restart plugsure-api plugsure-gateway` (Path A) or `docker compose up -d` (Path B). A gateway restart drops charger sockets; chargers reconnect within their backoff. Uncaught exceptions now exit the process, and systemd or Docker restarts it.

Retention

An hourly gateway pass deletes OCPP frames older than `OCPP_FRAME_RETENTION_DAYS` (90) and connection attempts older than `CONNECTION_ATTEMPT_RETENTION_DAYS` (30), in batches. Sessions, CDRs and the audit log are never touched. `0` keeps rows forever.

Rate limits

- Console and callers without a key: `API_RATE_LIMIT_PER_MIN` (600) per IP, per API process.
- API keys: `API_KEY_RATE_LIMIT_PER_MIN` (600) or the key's own limit, shared across API processes in PostgreSQL (`API_RATE_LIMIT_SHARED`).
- Keys that do not authenticate: `API_KEY_AUTH_FAILURES_PER_MIN` (30) per IP.
- Driver sign-in (OTP and PIN) limits per phone, IP, device, card and globally; refusals answer 429. See `RELEASE-NOTES-v1.4.0.md`.
- Per charger: `OCPP_MAX_MESSAGES_PER_S` (20) / `OCPP_MESSAGE_BURST` (200). Past it messages are paced, never dropped.

Routine

- Daily: dashboard alerts; `systemctl --failed` (backup); **Refunds**.
- Weekly: Compliance page; sessions flagged for review; held partner CDRs.
- Monthly: restore drill (section 10.2); **Availability** report.

- Per policy: rotate AuthorizationKeys from the charger's Security tab.

Feature set-up in the console (alert routing, platform billing, owners, roaming, fleet billing, integrations, Plug & Charge, white-label apps) is unchanged from v1.3: see section 12 of the v1.3 guide and `deploy/README.md`, together with the behaviour changes in 11.5. Console branding (v1.5.0) is in section 7.5.

13. Troubleshooting

13.1 A process refuses to start

The reason is in the first lines of `journalctl -u plugsure-api / -u plugsure-gateway` or `docker compose logs`. On Path A a failed migration fails the API or gateway start: its reason is in `journalctl -u plugsure-migrate`. After 5 failures in 5 minutes systemd stops retrying: fix the cause, then `systemctl reset-failed` and start again.

Message (start of)	Fix
AUDIT_HMAC_KEY is empty...	Set AUDIT_HMAC_KEY (openssl rand -hex 32).
SECRETS_KEY is not set / ...still the documentation placeholder / ...is too short	Set a 64-hex SECRETS_KEY. Changing an existing key means re-entering sealed credentials.
Refusing to start in NODE_ENV=...: ... (gateway)	OCPP_MIN_SECURITY_PROFILE=2 and OCPP_AUTO_ADOPT=false. Remember .env.example ships 0 and true.
OCPP_MIN_SECURITY_PROFILE=2 requires TLS...	Set OCPP_TRUST_PROXY_PROTO=true (TLS at Caddy).
The database user "..." is a superuser... / ...has BYPASSRLS...	Point DATABASE_URL at plugsure_app.
Could not determine whether row-level security constrains the database user...	Database unreachable or login failed: check DATABASE_URL, that POSTGRES_APP_PASSWORD was set when the migrator ran (Path A: in migrate.env), and URL-encoding of the password.
API_DEV_NO_AUTH is set but NODE_ENV is...	Remove API_DEV_NO_AUTH.
Compose: set POSTGRES_APP_PASSWORD in .env (or another set ... in .env)	Set the named variable in .env.
Migration: permission denied to create role	Owner lacks CREATEROLE: ALTER ROLE plugsure CREATEROLE;
Migration 006: permission denied to alter role	GRANT plugsure_app TO plugsure WITH ADMIN OPTION;
Migration: ...could not get a table lock within MIGRATION_LOCK_TIMEOUT...	Nothing applied. Find the long transaction in pg_stat_activity, then retry.
systemctl reload: "Job type reload is not applicable"	Expected. Use systemctl restart.

13.2 Running, but something is wrong

Symptom	Cause and fix
Every charger offline in the console; commands fail "not connected"	Bridge not configured: same INTERNAL_API_TOKEN on both, GATEWAY_INTERNAL_URL on the API, EVENT_RELAY=true on the gateway. API log must say "API bridge engaged".
Console 404 while /healthz works (Path A)	Web assets not copied: cp -R src/web dist/web.
Signed in, then logged out at once	Secure cookie over plain HTTP. Use HTTPS through Caddy; set API_TRUSTED_PROXIES.
Roaming partner gets 503	OCPI_PUBLIC_URL / PUBLIC_BASE_URL not set, or no roaming identity set up yet.
Alert e-mail to an internal SMTP relay refused	List it in SMTP_ALLOWED_INTERNAL_HOSTS.
Logs: ...holds UPDATE or DELETE on audit_log...	Revoke them, or re-run the migrator, which re-asserts the revokes.

13.3 A charger cannot connect

From `deploy/AUTEL-HANDOUT.md` and `deploy/README.md`. Ask the vendor for the identity and a UTC timestamp, and look the attempt up in **Connections** / `GET /v1/connection-attempts`.

Status	Meaning	Fix
101	Success: WebSocket open.	—
400	No supported subprotocol offered (e.g. a 2.0.1-only unit while <code>OCPP_VERSIONS=ocpp1.6</code>).	Set the unit to OCPP 1.6J. A unit that sends no subprotocol is taken as 1.6.
401	Missing or wrong Basic credentials.	Username must be the identity; password the AuthorizationKey. Re-issue or rotate the key.
403	No TLS at Profile 2 (<code>rejected_tls_required</code>).	Use <code>wss://</code> , not <code>ws://</code> . Check <code>OCPP_TRUST_PROXY_PROTO</code> and <code>OCPP_TRUSTED_PROXIES</code> .
404	Identity not registered: <i>unknown charge point — parked for adoption</i> ; or <i>charge point id missing from path</i> .	Register (section 8) or adopt it; check the URL ends with <code>/ocpp/<identity></code> and nobody added <code>handle_path</code> or a rewrite.
Pending at BootNotification	Authenticated but not activated, or suspended.	<code>POST .../activate</code> , or Resume .

Other patterns: nothing in the Caddy access log → DNS, SIM/APN routing or firewall. TLS handshake fails → ISRG Root X1 missing in the charger; never drop to `ws://` . Drops every ~5 minutes → a proxy timeout below 900 s in the path.

14. Go-live checklist

- ☐ Server runs `v1.5.0`; `NODE_ENV=production`; `TZ=Asia/Jakarta`
- ☐ Secrets generated and stored in the password manager, separate from backups; `env` file is `0640 root:pluginsure`, `migrate.env` and `backup.env` are `0600 root:root` and `pluginsure.env` holds no owner credential (Path A), or `.env` is mode 600 (Path B)
- ☐ `OCPP_MIN_SECURITY_PROFILE=2`, `OCPP_AUTO_ADOPT=false`, `OCPP_TRUST_PROXY_PROTO=true`; `ALLOW_INSECURE_OCPCP` not set
- ☐ Charger hostname final; Let's Encrypt production certificate issued (not staging); ISRG Root X1 confirmed with the vendor
- ☐ Ports 9200, 9220 and 5432 not reachable from outside; console host behind the office allow-list
- ☐ Both processes log "row-level security is in force for application queries"; the API logs "API bridge engaged"
- ☐ `/healthz` on 9200 and 9220 returns `"ok":true`; `https://<ocpp host>/ocpp/test` returns 426
- ☐ `GET /v1/audit` shows `"chain": {"ok": true, ...}` with no problems
- ☐ First administrator created and password changed; team invited with least-privilege roles
- ☐ `deploy/Caddyfile v1.4` deployed; `api.` host (if kept) refuses `/v1/auth/login`
- ☐ (Optional, v1.5.0) White-label console addresses: each has its own Caddy site block with that operator's allow-list, and is approved by a platform administrator; `CONSOLE_PUBLIC_URL` names the shared console address (section 7.5)
- ☐ Nightly backup timer enabled, `BACKUP_OFFSITE_CMD` set, one restore drill done and timed
- ☐ First charger registered, keyed, at Profile 2, activated; `deploy/AUTEL-HANDOUT.md` sent, key sent by a separate channel
- ☐ Tax settings and tariffs confirmed with the finance team and a tax adviser
- ☐ **Run the v1.5.0 Hardware Acceptance Test on at least one real charger:** `docs/PlugSure-v1.5.0-Hardware-Acceptance-Test.pdf` (printable, from `docs/ACCEPTANCE-v1.4.md`). Every **Pass** must hold and the sheet must be signed off. This is the go-live gate: one failed **Pass** stops the pilot until the failure is explained.

PlugSure CSMS v1.5.0 · Deployment Guide · 2 October 2026 · See also `RELEASE-NOTES-v1.4.0.md`, `RELEASE-NOTES-v1.4.1.md`, `RELEASE-NOTES-v1.4.2.md`, `RELEASE-NOTES-v1.4.3.md`, `RELEASE-NOTES-v1.4.4.md`, `RELEASE-NOTES-v1.5.0.md`, `deploy/README.md`, `deploy/AUTEL-HANDOUT.md` and `docs/ACCEPTANCE-v1.4.md`.